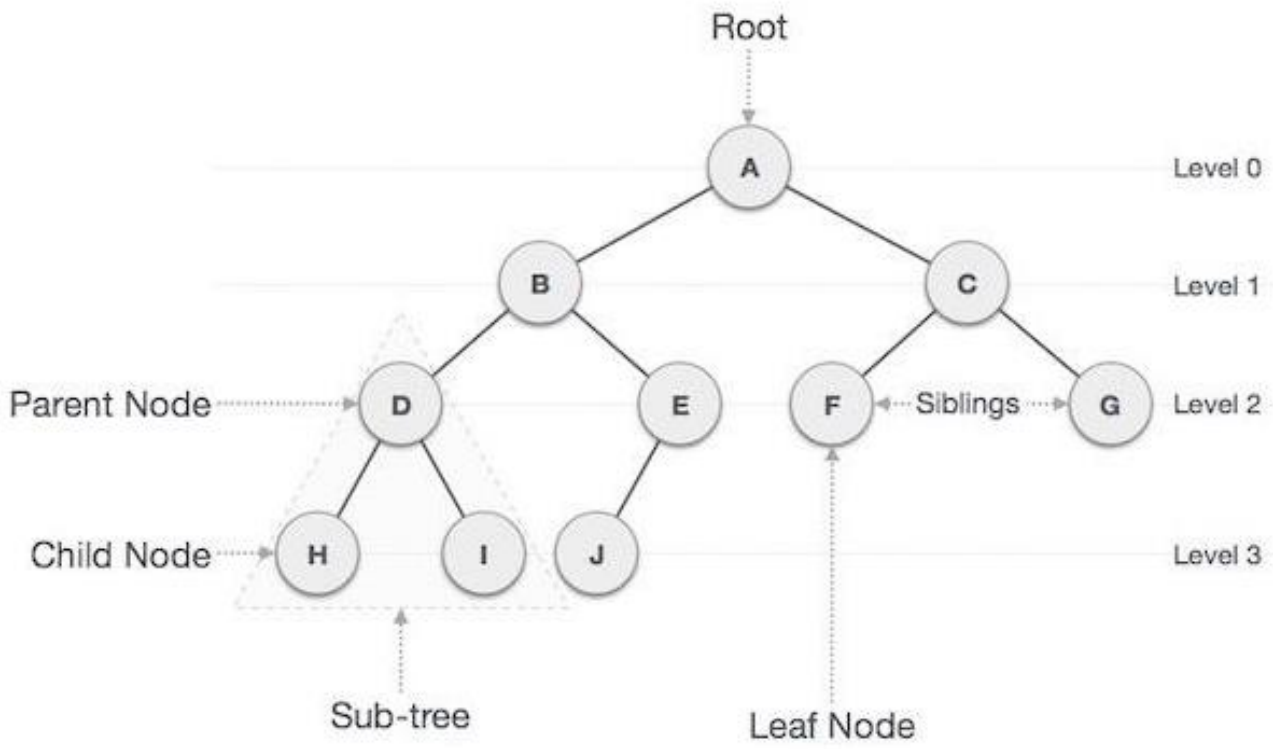
Trees

Trees terminologies

- Degree
- Level
- In degree
- out degree
- Height
- path
- Leaf
- Ancestors
- descendent

Tree Data Structure

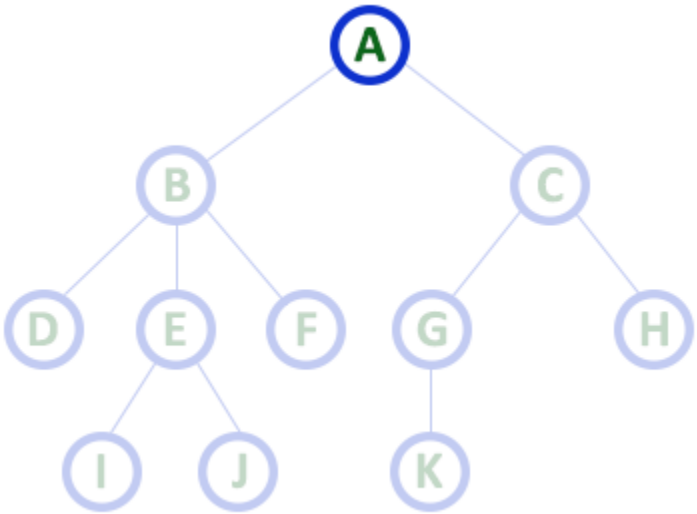
- A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges.


Root
A
Level 0
B
C
Level 1
Parent Node
D
E
F
Siblings
G
Level 2
Child Node
H
I
J
Level 3
Sub-tree
Leaf Node

Tree Terminologies

- **Node**
  - A node is an entity that contains a key or value and pointers to its child nodes.
  - The last nodes of each path are called **leaf nodes or external nodes** that do not contain a link/pointer to child nodes.
  - The node having at least a child node is called an **internal node**.

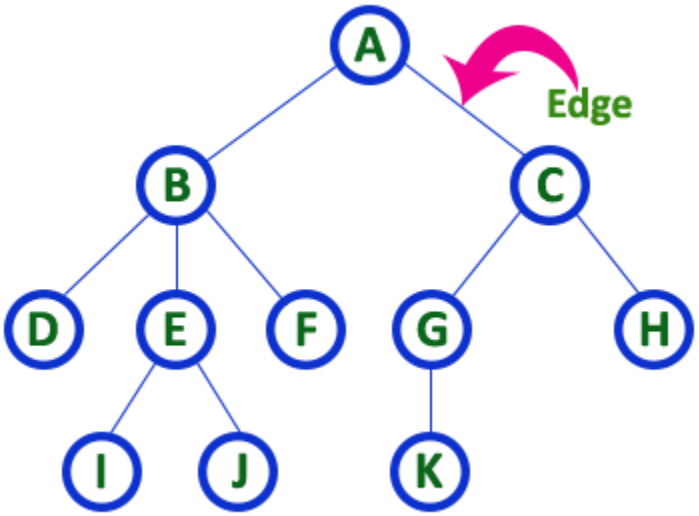
- **1. Root**
- In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. We can say that the root node is the origin of the tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.
-



Here 'A' is the 'root' node

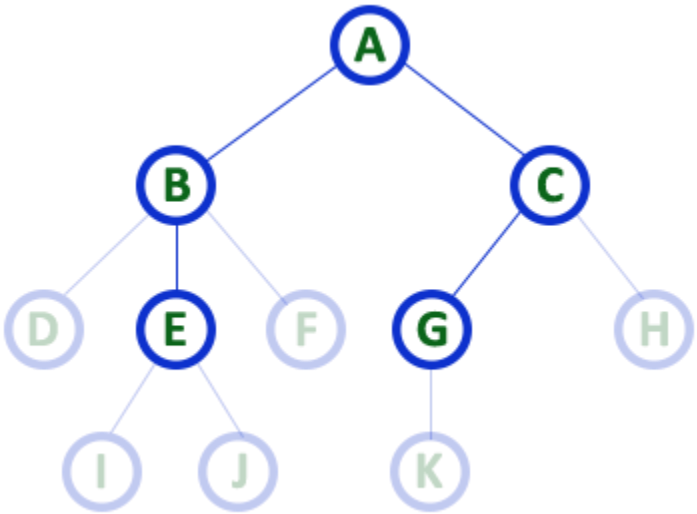
- In any tree the first node is called as ROOT node

- **2. Edge**
- In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.



- In any tree, 'Edge' is a connecting link between two nodes.

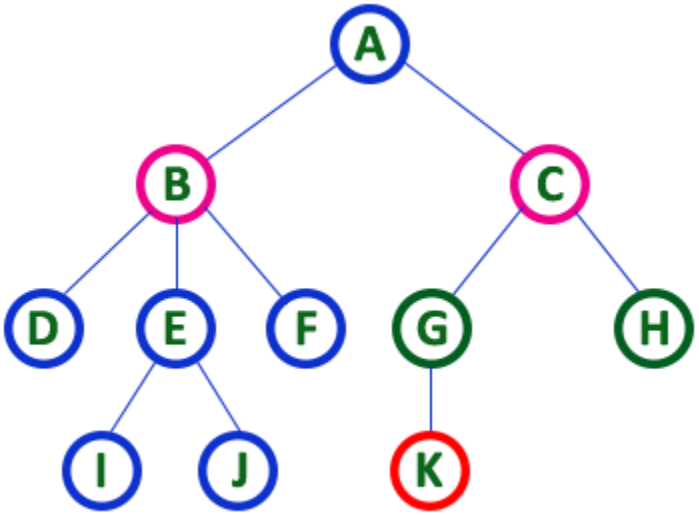
- **3.Parent**
- In a tree data structure, the node which is a predecessor of any node is called as **PARENT NODE**. In simple words, the node which has a branch from it to any other node is called a parent node. Parent node can also be defined as "**The node which has child / children**".



Here A, B, C, E & G are Parent nodes

- **In any tree the node which has child / children is called 'Parent'**
- **A node which is predecessor of any other node is called 'Parent'**

- **Child**
- In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.



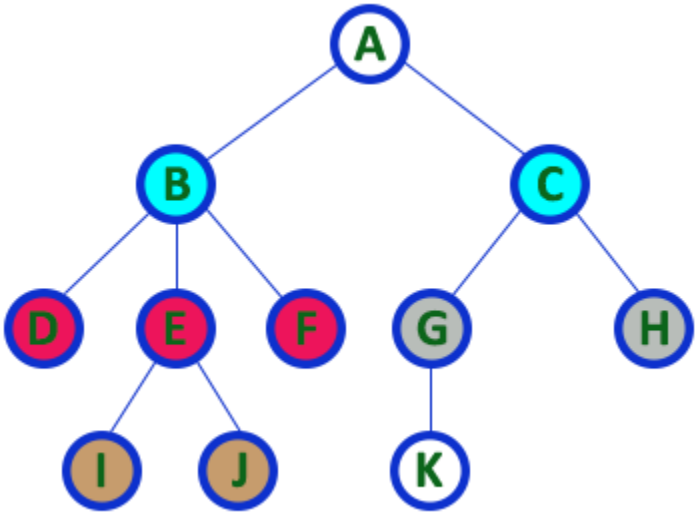
Here B & C are Children of A

Here G & H are Children of C

Here K is Child of G

- descendant of any node is called as CHILD Node

- **Siblings**
- In a tree data structure, nodes which belong to same Parent are called as **SIBLINGS**. In simple words, the nodes with the same parent are called Sibling nodes.



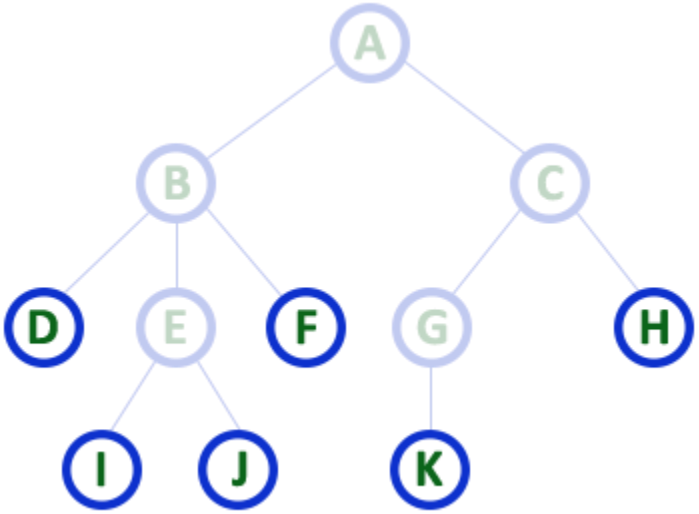
Here B & C are Siblings
Here D E & F are Siblings
Here G & H are Siblings
Here I & J are Siblings

- In any tree the nodes which has same Parent are called 'Siblings'

- The children of a Parent are called 'Siblings'

- **Leaf**
- In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child.

  In a tree data structure, the leaf nodes are also called as **External Nodes**. External node is also a node with no child. In a tree, leaf node is also called as '**Terminal**' node.

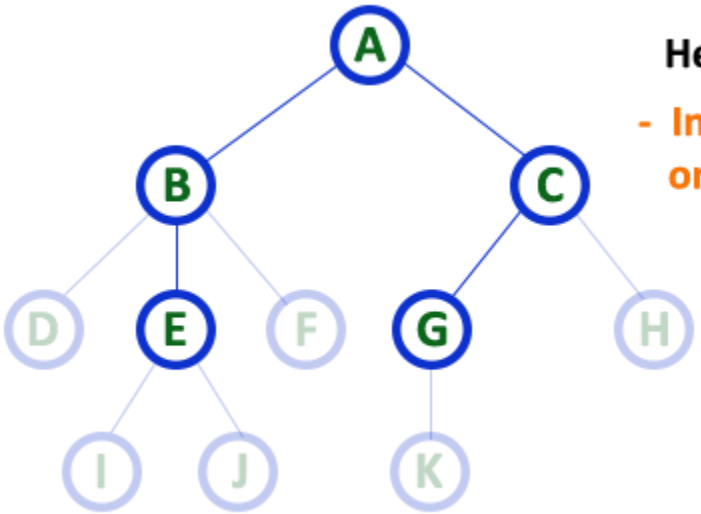


Here D, I, J, F, K & H are Leaf nodes

- **In any tree the node which does not have children is called 'Leaf'**
- **A node without successors is called a 'leaf' node**

- **Internal Nodes**
- In a tree data structure, the node which has atleast one child is called as **INTERNAL Node**. In simple words, an internal node is a node with atleast one child.

  In a tree data structure, nodes other than leaf nodes are called as **Internal Nodes**. **The root node is also said to be Internal Node** if the tree has more than one node. Internal nodes are also called as '**Non-Terminal**' nodes.



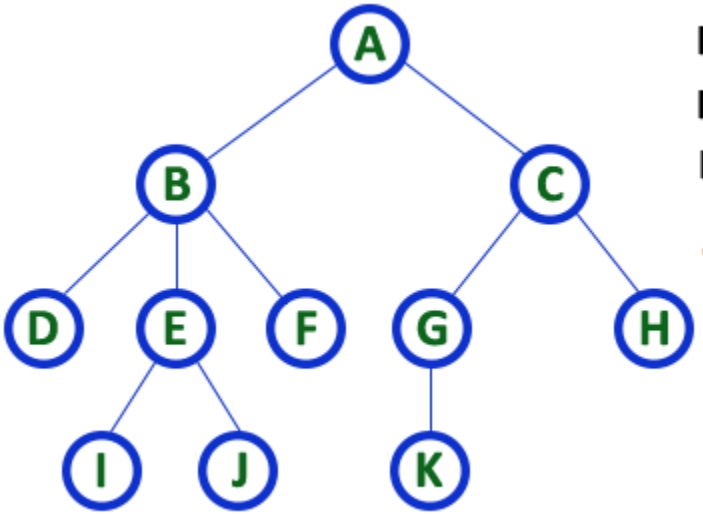
Here A, B, C, E & G are Internal nodes

- **In any tree the node which has atleast one child is called 'Internal' node**

- **Every non-leaf node is called as 'Internal' node**

- **Degree**
- In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'
-

- **In-degree:** Represents incoming edges. In a rooted tree, every node except the root has an in-degree of 1, as each child has only one parent.
- **Out-degree:** Represents outgoing edges, corresponding to the number of children a node has.



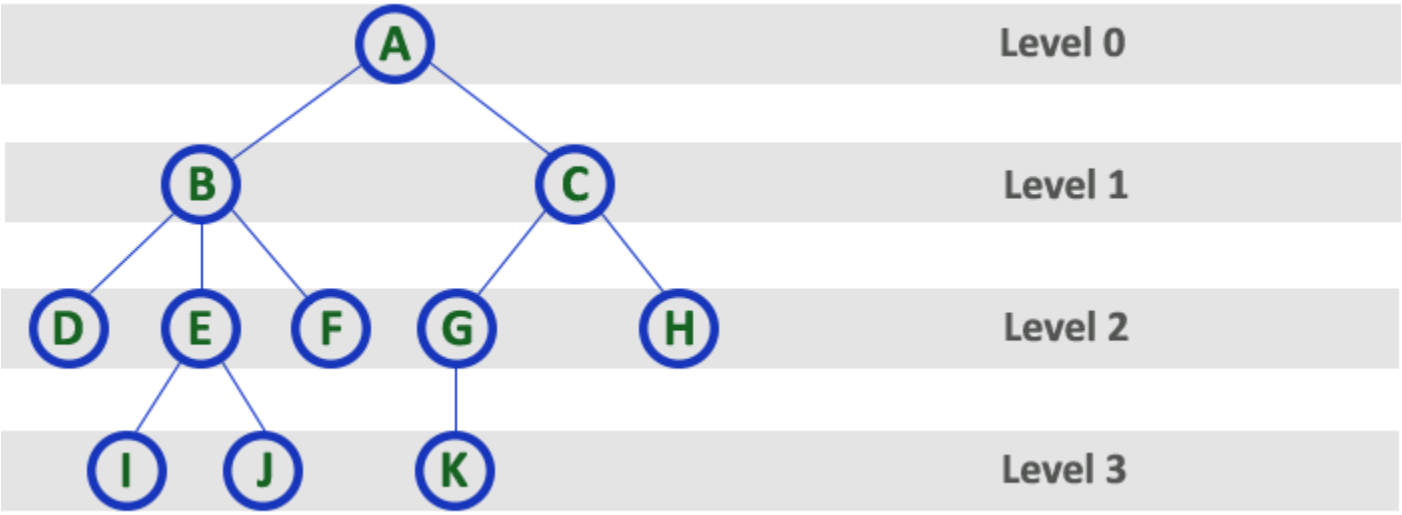
Here Degree of B is 3

Here Degree of A is 2

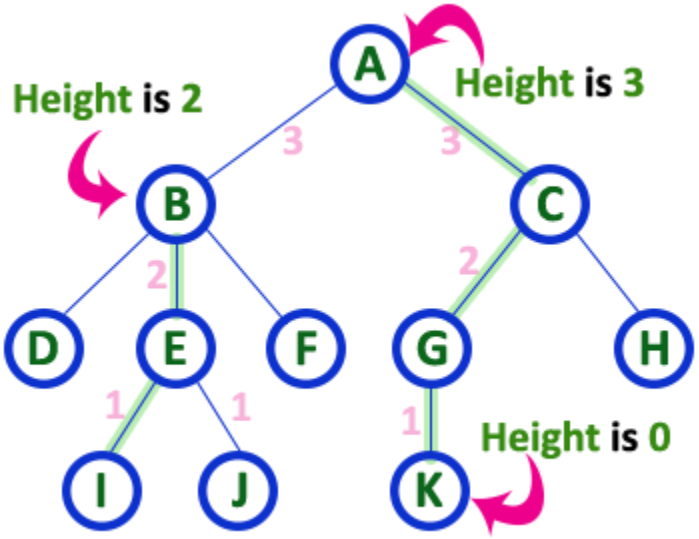
Here Degree of F is 0

- In any tree, 'Degree' of a node is total number of children it has.

- **Level**
- In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).


A
Level 0
B
C
Level 1
D
E
F
G
H
Level 2
I
J
K
Level 3

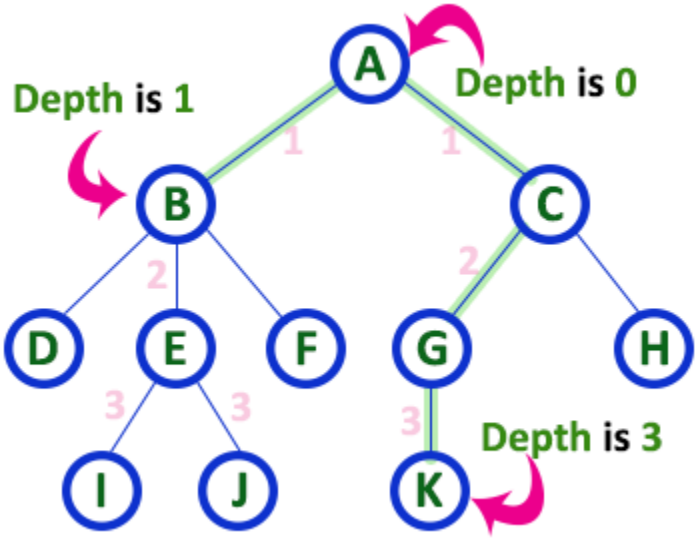
- **Height**
- In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node. In a tree, height of the root node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'.**



Here Height of tree is 3

- In any tree, 'Height of Node' is total number of Edges from leaf to that node in longest path.
- In any tree, 'Height of Tree' is the height of the root node.

- **Depth**
- In a tree data structure, the total number of egdes from root node to a particular node is called as **DEPTH** of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**. In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree. In a tree, **depth of the root node is '0'.**

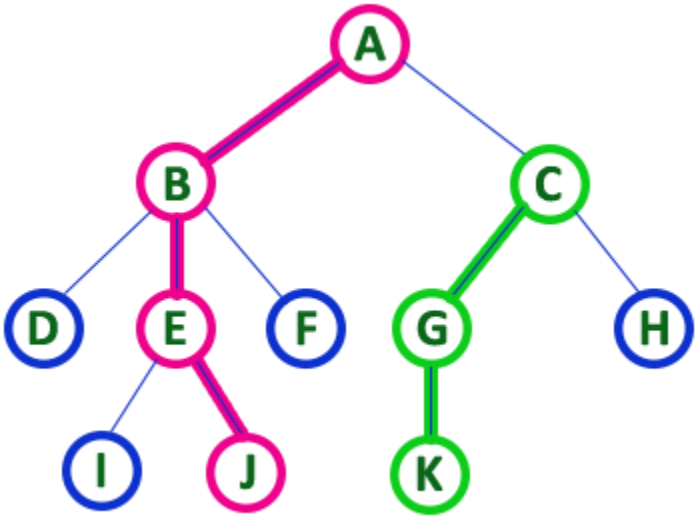


Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.

- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

- **Path**
- In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes. **Length of a Path** is total number of nodes in that path. In below example **the path A - B - E - J has length 4**.



- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

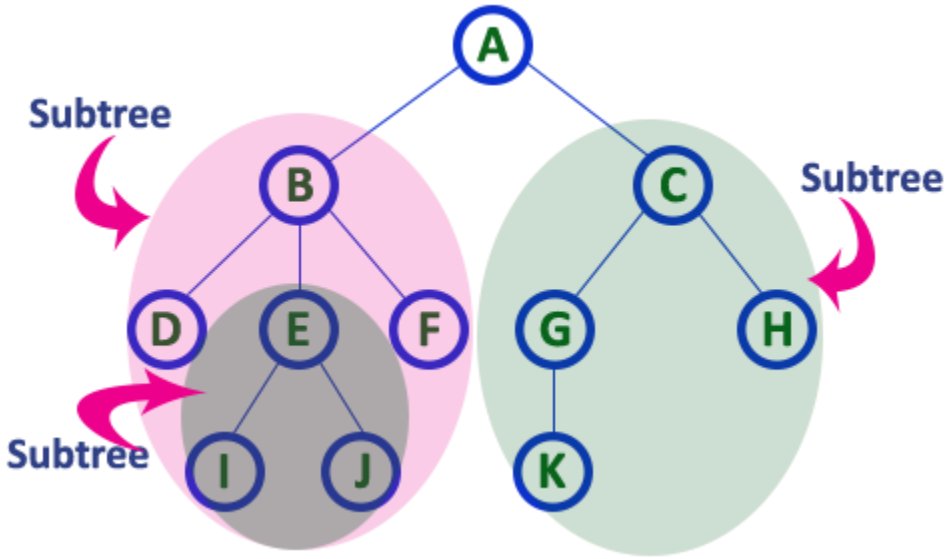
Here, 'Path' between A & J is

A - B - E - J

Here, 'Path' between C & K is

C - G - K

- **Sub Tree**
- In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.

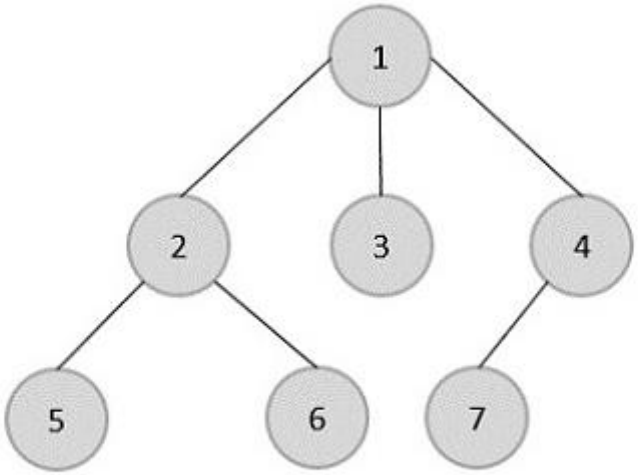

A
Subtree
B
C
Subtree
D
E
F
G
H
Subtree
I
J
K

Types of trees

1. General tree
2. Binary tree
3. Binary search tree

General Trees

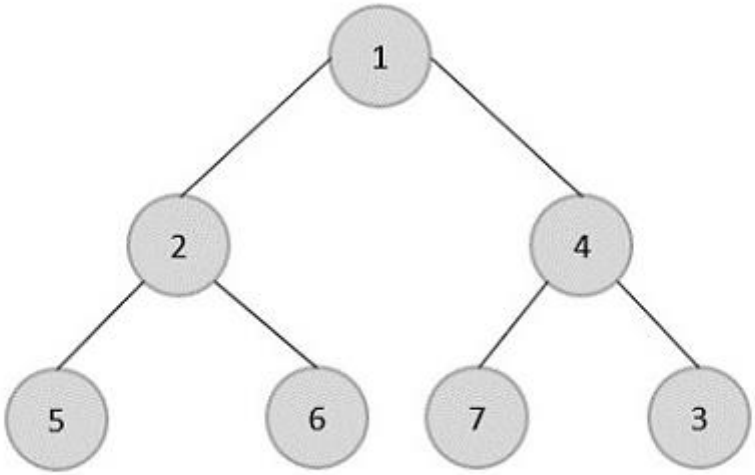
- General trees are unordered tree data structures where the root node has minimum 0 or maximum n subtrees.
- The General trees have no constraint placed on their hierarchy.
- The root node thus acts like the superset of all the other subtrees.



General Tree Data Structure

Binary Trees

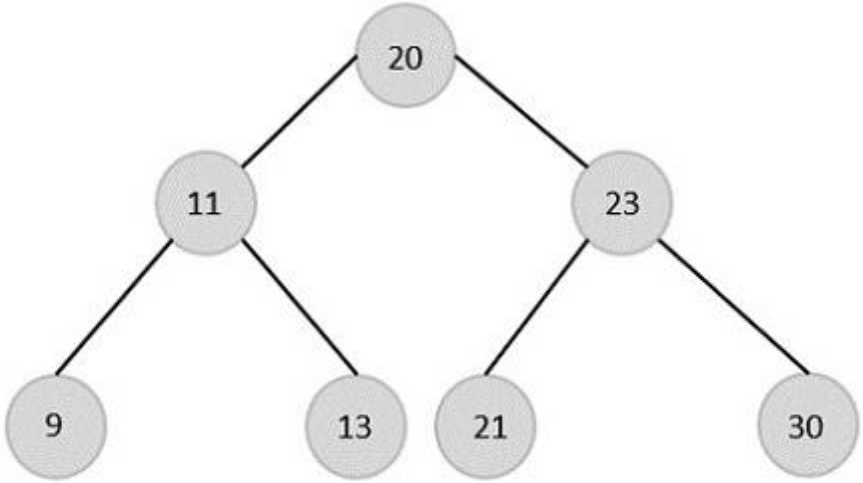
- Binary Trees are general trees in which the root node can only hold up to maximum 2 subtrees: left subtree and right subtree.
- In a binary tree, every node can have either 0 children or 1 child or 2 children but not more than 2 children.
- Based on the number of children, binary trees are divided into three types.
  - Full
  - Complete
  - Perfect



Binary Tree Data Structure

Binary Search Trees

- Binary Search Trees possess all the properties of Binary Trees including some extra properties of their own, based on some constraints, making them more efficient than binary trees.
- The data in the Binary Search Trees (BST) is always stored in such a way that the values in the left subtree are always less than the values in the root node and the values in the right subtree are always greater than the values in the root node, i.e. left subtree < root node right subtree.



Binary Search Tree Data Structure